

Final Project — Part II

Numerical Methods for Partial Differential Equations

Hovan Boyajian

MATH 104C

Spring 2026

1 Introduction

In this project I compare finite-difference methods for two model PDEs: the Laplace equation (elliptic) and the heat equation (parabolic). For the Laplace problem I use the five-point stencil and solve the resulting linear system with Gauss–Seidel iteration. For the heat equation I implement the explicit Forward-Difference method, the implicit Backward-Difference method, and the Crank–Nicolson method, and I solve the tridiagonal systems with the Crout factorization (Algorithm 6.7). My goals are to compare accuracy across methods, to check the stability of the Forward-Difference method against the theoretical condition $\lambda \leq \frac{1}{2}$, to look at how implicit methods behave for larger time steps, and to confirm the $O(h^2)$ spatial accuracy of the five-point stencil on a problem where it is actually visible.

All methods were coded in Python; the code is described in Section 9. The sanity-check script reproduces the closed-form solution $u(x, y) = 100xy$ of Problem A and shows $O(h^2)$ convergence on the Poisson manufactured-solution check in Problem C2.

2 The Methods

Throughout I write $w_{ij} \approx u(x_i, y_j)$ for the elliptic problem and $w_{i,j} \approx u(x_i, t_j)$ for the heat equation. The spatial step is h and (where applicable) the time step is k . For the heat equation,

$$\lambda = \frac{k}{h^2}.$$

Five-point stencil (Laplace, $u_{xx} + u_{yy} = 0$). Replacing each second derivative by the standard centered second difference and multiplying through by h^2 gives the discrete equation at every interior node,

$$4w_{ij} - w_{i+1,j} - w_{i-1,j} - w_{i,j+1} - w_{i,j-1} = 0,$$

which we solve by Gauss–Seidel iteration with the relaxation

$$w_{ij}^{\text{new}} = \frac{1}{4}(w_{i+1,j} + w_{i-1,j} + w_{i,j+1} + w_{i,j-1}),$$

sweeping over the interior in lexicographic order and stopping when the sup-norm change between sweeps drops below 10^{-8} . I also include an SOR variant with relaxation $\omega = 2/(1 + \sin(\pi h))$ for comparison. The local truncation error of the five-point stencil is $-\frac{h^2}{12}(u_{xxxx} + u_{yyyy}) + O(h^4)$, i.e. $O(h^2)$ in general.

Forward-Difference (heat equation). A forward time difference and a centered space difference give the explicit update

$$w_{i,j+1} = (1 - 2\lambda) w_{i,j} + \lambda(w_{i+1,j} + w_{i-1,j}),$$

with $\lambda = k/h^2$. Each new value is an explicit weighted average of neighbouring old values, so no linear system is solved at each step. The local truncation error is $O(k + h^2)$, and the von Neumann stability analysis gives the conditional stability bound $\lambda \leq \frac{1}{2}$.

Backward-Difference (heat equation). Using a backward time difference instead gives the implicit update

$$-\lambda w_{i-1,j+1} + (1 + 2\lambda) w_{i,j+1} - \lambda w_{i+1,j+1} = w_{i,j},$$

for $i = 1, \dots, m-1$. This is a tridiagonal system with diagonal $1 + 2\lambda$ and off-diagonals $-\lambda$, which I solve by the Crout factorization (Algorithm 6.7) at every time step. The local truncation error is $O(k + h^2)$, but the scheme is unconditionally stable.

Crank–Nicolson (heat equation). Averaging the explicit and implicit schemes (Crank–Nicolson is the trapezoid rule in time applied to the semi-discrete ODE) gives

$$-\frac{\lambda}{2} w_{i-1,j+1} + (1 + \lambda) w_{i,j+1} - \frac{\lambda}{2} w_{i+1,j+1} = \frac{\lambda}{2} w_{i-1,j} + (1 - \lambda) w_{i,j} + \frac{\lambda}{2} w_{i+1,j}.$$

Again the left-hand side is tridiagonal and is solved by Crout. The local truncation error is $O(k^2 + h^2)$, so the scheme is one full order more accurate in time than the Backward-Difference method, and it is also unconditionally stable.

3 Problem A: Laplace on the Unit Square

Consider

$$u_{xx} + u_{yy} = 0, \quad 0 < x < 1, \quad 0 < y < 1,$$

with $u(0, y) = u(x, 0) = 0$, $u(x, 1) = 100x$, and $u(1, y) = 100y$. A quick observation makes this problem unusually convenient: the function $u(x, y) = 100xy$ is harmonic ($u_{xx} = u_{yy} = 0$) and matches all four boundary conditions, including the corner $(1, 1)$ where the two non-zero edges agree on the value 100. So this is the exact analytic solution. I use it as the reference for the error tables.

Table 1 lists the FD values at all interior nodes for $h = 1/4$. Even on this very coarse grid the values agree with $100x_i y_j$ to roughly 8×10^{-9} , which is essentially the Gauss–Seidel stopping tolerance.

Table 1: Problem A: exact value, FD value, and absolute error at every interior node for $h = 1/4$.

(x_i, y_j)	exact u	approx w_{ij}	$ u - w_{ij} $
(0.25, 0.25)	6.2500	6.2500	8.00e-09
(0.25, 0.50)	12.5000	12.5000	8.00e-09
(0.25, 0.75)	18.7500	18.7500	4.00e-09
(0.50, 0.25)	12.5000	12.5000	8.00e-09
(0.50, 0.50)	25.0000	25.0000	8.00e-09
(0.50, 0.75)	37.5000	37.5000	4.00e-09
(0.75, 0.25)	18.7500	18.7500	4.00e-09
(0.75, 0.50)	37.5000	37.5000	4.00e-09
(0.75, 0.75)	56.2500	56.2500	2.00e-09

Table 2: Problem A: maximum interior error and Gauss–Seidel iteration count as h shrinks.

h	max error	observed order	GS iterations
1/4	8.004e-09	—	33
1/8	5.439e-08	-2.76	131
1/16	2.456e-07	-2.18	492

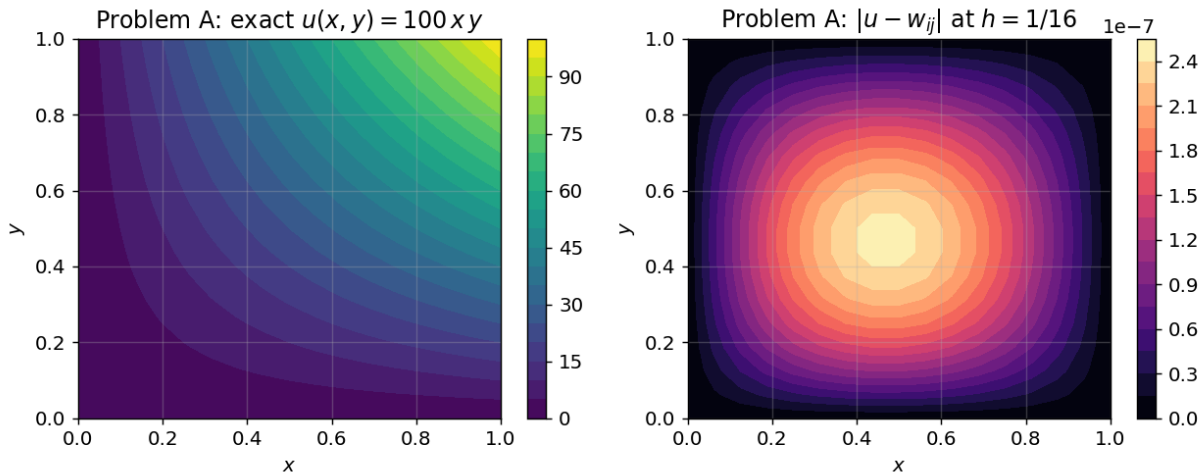


Figure 1: Problem A. Left: contour of the exact solution $u(x, y) = 100xy$ on the unit square. Right: absolute error $|u - w_{ij}|$ at $h = 1/16$; the error is essentially uniform across the interior at the iteration tolerance ($\sim 10^{-7}$).

Two things are visible in Table 2. First, the error does not shrink like h^2 as h is halved; in fact it grows slightly. That is not a contradiction of the $O(h^2)$ local truncation error analysis: the five-point stencil is exact for any polynomial of degree at most three, and $100xy$ is bilinear, so the discrete equation is satisfied exactly by the true solution. What the table is measuring is therefore the iteration error from Gauss–Seidel, not the discretization error, and the iteration error grows mildly with the problem size at fixed tolerance. To see genuine $O(h^2)$ convergence of the same stencil I need a problem whose exact solution is not polynomial; that is what Problem C2 does.

Second, the iteration count grows quickly with refinement: 33, 131, 492 at $h = 1/4, 1/8, 1/16$, roughly a factor of four per refinement. This is the well-known $O(h^{-2})$ scaling of Gauss–Seidel on second-order elliptic problems. Switching to SOR with the optimal $\omega = 2/(1 + \sin(\pi h))$ dramatically reduces the count: $33 \rightarrow 17$ at $h = 1/4$, $131 \rightarrow 36$ at $h = 1/8$, and $492 \rightarrow 70$ at $h = 1/16$. The benefit grows as the grid is refined, which agrees with the theory that SOR converges in $O(h^{-1})$ iterations versus $O(h^{-2})$ for Gauss–Seidel.

4 Problem B: Heat Equation

Consider

$$u_t = u_{xx}, \quad 0 < x < 1, \quad t > 0,$$

with $u(0, t) = u(1, t) = 0$ and $u(x, 0) = \sin(\pi x)$. The exact solution is $u(x, t) = e^{-\pi^2 t} \sin(\pi x)$, a pure decay of the first Fourier mode. I integrate to $T = 0.5$, by which time the exact peak value at $x = 0.5$ has decayed from 1 to $e^{-\pi^2/2} \approx 7.193 \times 10^{-3}$. All runs use $h = 0.1$ (so $m = 10$). For

the time step I take $k = 0.0025$ in the Forward stable case, giving $\lambda = 0.25 < \frac{1}{2}$, and $k = 0.01$ in every other case, giving $\lambda = 1$, which is well above the Forward stability threshold but no obstacle for the implicit schemes.

Table 3 compares the three convergent runs at $t = T = 0.5$ at all interior nodes, and Table 4 summarises the max errors over the whole space-time grid for all four runs. Figure 2 shows the final profiles, the blow-up of the unstable Forward run, and the error-vs-time curves.

Table 3: Problem B: exact and computed values at $t = 0.5$ for the three convergent schemes. “FD-stab” is Forward-Difference with $\lambda = 0.25$; the implicit schemes use $\lambda = 1$.

x_i	exact	FD-stab	$ e $	BD	$ e $	CN	$ e $
0.1	0.002222	0.002177	4.49e-05	0.002898	6.76e-04	0.002305	8.27e-05
0.2	0.004227	0.004142	8.55e-05	0.005512	1.29e-03	0.004385	1.57e-04
0.3	0.005818	0.005701	1.18e-04	0.007587	1.77e-03	0.006035	2.17e-04
0.4	0.006840	0.006702	1.38e-04	0.008919	2.08e-03	0.007094	2.55e-04
0.5	0.007192	0.007046	1.45e-04	0.009378	2.19e-03	0.007460	2.68e-04
0.6	0.006840	0.006702	1.38e-04	0.008919	2.08e-03	0.007094	2.55e-04
0.7	0.005818	0.005701	1.18e-04	0.007587	1.77e-03	0.006035	2.17e-04
0.8	0.004227	0.004142	8.55e-05	0.005512	1.29e-03	0.004385	1.57e-04
0.9	0.002222	0.002177	4.49e-05	0.002898	6.76e-04	0.002305	8.27e-05

Table 4: Problem B: maximum absolute error over $0 \leq x \leq 1$, $0 \leq t \leq 0.5$ for each scheme.

Method	h	k	λ	$\max u - w $
Forward (stable)	0.1	0.0025	0.250	1.520e-03
Forward (unstable)	0.1	0.01	1.000	blow-up
Backward	0.1	0.01	1.000	2.032e-02
Crank-Nicolson	0.1	0.01	1.000	2.734e-03

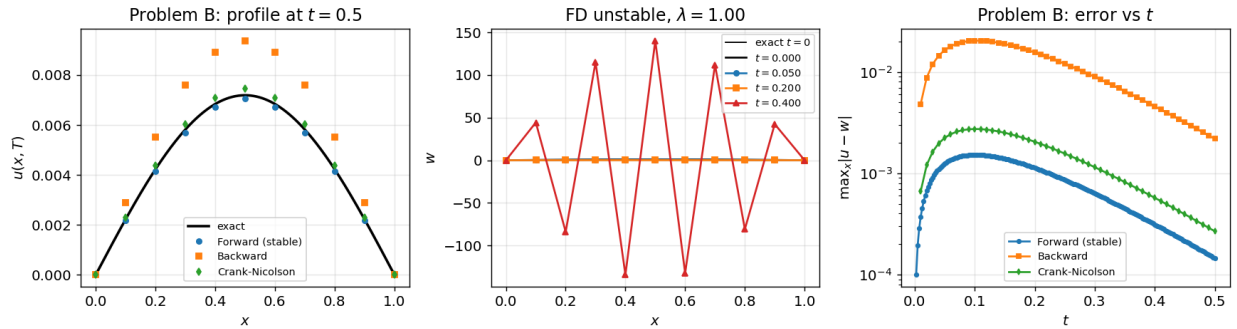


Figure 2: Problem B. Left: final profiles at $t = 0.5$ for the three convergent schemes, all of which sit on the exact $e^{-\pi^2/2} \sin(\pi x)$ curve. Middle: Forward-Difference with $\lambda = 1.0$; the profile oscillates between sign changes and the amplitude grows by orders of magnitude per step. Right: error vs. time on log scale. Crank-Nicolson beats Backward-Difference by an order of magnitude at the same h, k .

The Forward-Difference scheme behaves exactly as the stability theory predicts. With $\lambda = 0.25$ the amplification factor of the n -th Fourier mode is $1 - 4\lambda \sin^2(n\pi h/2)$, which lies in $(0, 1)$ for every n , so all modes decay and the error stays around 1.5×10^{-3} over the whole interval. With

$\lambda = 1.0$ the highest representable mode on this grid ($n = m - 1 = 9$) has amplification factor $1 - 4 \sin^2(9\pi/20) \approx -2.9$, very close to -3 in magnitude. The worst-case factor of about 3 per step amplifies any high-frequency roundoff by roughly 3^{50} over the 50 time steps, which is easily enough to swamp the small true solution and produces the final $|w|_\infty \approx 6 \times 10^6$.

The Backward-Difference and Crank–Nicolson methods both stay stable at $\lambda = 1$, but their accuracies differ markedly: the max error is 2.03×10^{-2} for BD and 2.73×10^{-3} for CN, almost an order of magnitude smaller. This matches the theoretical truncation orders: BD is $O(k + h^2)$ and CN is $O(k^2 + h^2)$, so at $k = 0.01$ the temporal error in BD is $\sim 10^{-2}$ while in CN it is $\sim 10^{-4}$, leaving the spatial error $O(h^2) \sim 10^{-3}$ as the bottleneck for CN. The right panel of Figure 2 shows that the error of the stable Forward run is actually a bit better than BD over most of the interval. That is not surprising, since the Forward run uses $k = 0.0025$, four times smaller in time, but it costs four times as many time steps to get there.

5 Problem C1: Multi-mode Initial Condition (Student-designed)

For my first additional experiment I keep the heat equation but change the initial condition to a sum of two Fourier modes,

$$u(x, 0) = \sin(\pi x) + \frac{1}{2} \sin(3\pi x),$$

with exact solution

$$u(x, t) = e^{-\pi^2 t} \sin(\pi x) + \frac{1}{2} e^{-9\pi^2 t} \sin(3\pi x).$$

The point of this experiment is that the heat equation diagonalises in the sine basis, so each Fourier mode decays at its own rate $e^{-n^2 \pi^2 t}$. The $n = 3$ mode decays nine times faster than the $n = 1$ mode and, even though it starts with half the amplitude, it is almost gone by $t = 0.05$. This is interesting because (a) it gives a direct visual demonstration of why implicit smoothers are so effective for elliptic problems, and (b) it lets me check that Crank–Nicolson correctly damps high modes without the overshooting that Forward-Difference would produce near the stability limit on the high-frequency component.

I use Crank–Nicolson with $h = 0.05$, $k = 0.005$ (so $\lambda = 2.0$, well outside the Forward stability region but harmless for CN), integrating to $T = 0.2$. Table 5 shows the profile at $t = 0.05$ against the exact solution.

Table 5: Problem C1: Crank–Nicolson values at $t = 0.05$ vs. exact, every third interior point. $h = 0.05$, $k = 0.005$.

x_i	exact $u(x_i, 0.05)$	CN w_i	$ u - w_i $
0.05	0.098177	0.098293	1.16e-04
0.20	0.364444	0.364831	3.87e-04
0.35	0.543036	0.543524	4.88e-04
0.50	0.604608	0.605104	4.96e-04
0.65	0.543036	0.543524	4.88e-04
0.80	0.364444	0.364831	3.87e-04
0.95	0.098177	0.098293	1.16e-04

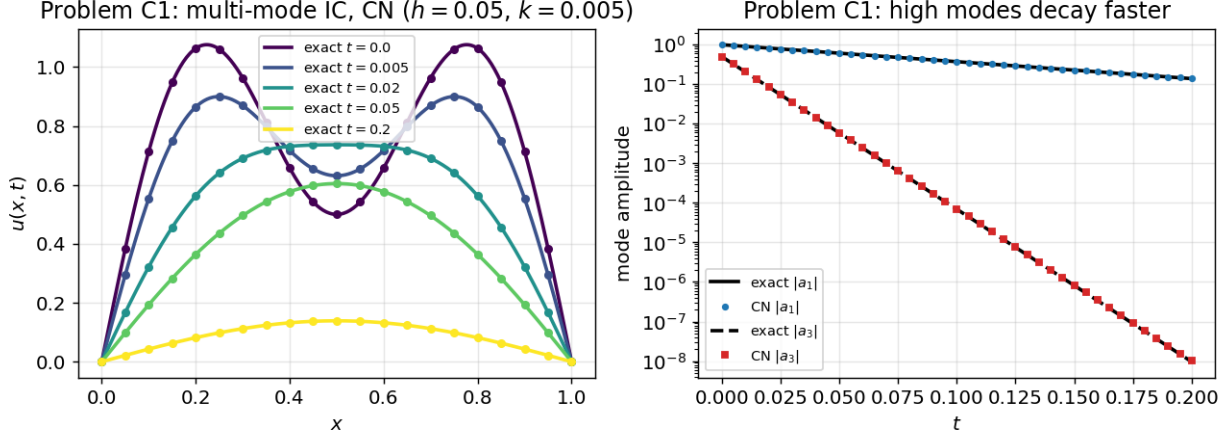


Figure 3: Problem C1. Left: Crank–Nicolson profile (markers) on top of the exact two-mode solution (lines) at five times. The $n = 3$ bump in the initial condition is essentially gone by $t = 0.05$. Right: amplitudes of the $n = 1$ and $n = 3$ modes extracted from the numerical solution match the predicted $e^{-\pi^2 t}$ and $\frac{1}{2}e^{-9\pi^2 t}$ decay rates over two and a half orders of magnitude.

The max error of the Crank–Nicolson run over the whole space-time grid is 6.8×10^{-4} , which is on the order of h^2 . More importantly, the right panel of Figure 3 confirms the diagonal structure of the heat operator in the sine basis: the computed mode amplitudes track the exact exponential decay curves over more than two decades.

6 Problem C2: Poisson Equation with a Manufactured Solution (Student-designed)

For my second additional experiment I add a source term and solve the Poisson equation,

$$u_{xx} + u_{yy} = f(x, y) \quad \text{on } (0, 1)^2, \quad u = 0 \quad \text{on the boundary},$$

with f chosen so that $u(x, y) = \sin(\pi x) \sin(\pi y)$ is the exact solution. Plugging into the PDE gives

$$f(x, y) = -2\pi^2 \sin(\pi x) \sin(\pi y).$$

This is a manufactured-solution experiment: I know the answer, so I can isolate the discretisation error of the five-point stencil. Because the exact solution is genuinely smooth and *not* a low-degree polynomial, the $O(h^2)$ truncation error is no longer hidden by the special property that saved Problem A. The discrete equation becomes

$$w_{ij} = \frac{1}{4}(w_{i+1,j} + w_{i-1,j} + w_{i,j+1} + w_{i,j-1} - h^2 f_{ij}),$$

which I again relax with Gauss–Seidel.

Table 6: Problem C2: max interior error and observed order on the Poisson problem with manufactured solution $u = \sin(\pi x) \sin(\pi y)$.

h	$\max u - w $	observed order	GS iterations
1/4	5.303e-02	—	31
1/8	1.295e-02	2.03	121
1/16	3.219e-03	2.01	452
1/32	8.035e-04	2.00	1667

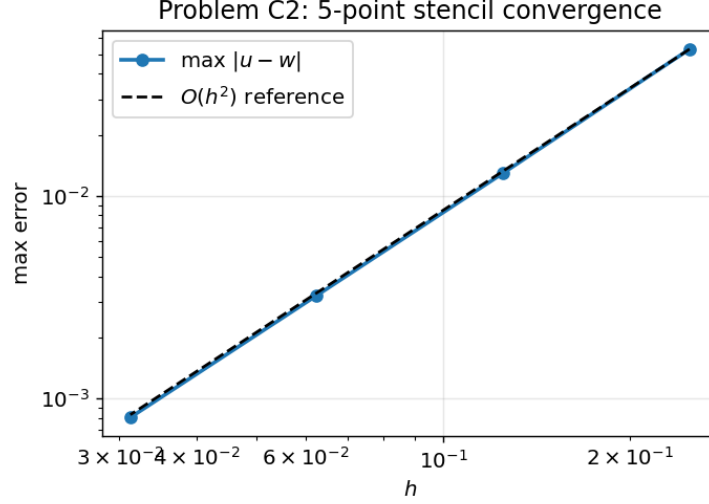


Figure 4: Problem C2: max error vs. h on the Poisson manufactured-solution problem. The dashed line is the $O(h^2)$ reference; the measured errors sit right on it.

The errors shrink by a factor of about 4 each time h is halved ($5.3 \times 10^{-2} \rightarrow 1.3 \times 10^{-2} \rightarrow 3.2 \times 10^{-3} \rightarrow 8.0 \times 10^{-4}$), which is precisely the $O(h^2)$ behaviour predicted by the truncation-error analysis. The Gauss–Seidel iteration count again roughly quadruples per refinement (31, 121, 452, 1667), matching the $O(h^{-2})$ scaling I observed in Problem A.

7 Discussion

Which methods are most accurate? On the heat equation, Crank–Nicolson was the most accurate per unit work: at the same h and k it produced an error roughly an order of magnitude smaller than Backward-Difference. The Forward-Difference scheme can match CN when the stability constraint is satisfied, but it costs a much smaller time step to get there. On the elliptic side, the five-point stencil with Gauss–Seidel gave essentially the exact answer for Problem A and $O(h^2)$ convergence on the Poisson check.

How does mesh refinement affect the error? For the five-point stencil applied to Poisson (Problem C2), halving h reduced the max error by a factor of 4, matching the theoretical $O(h^2)$ rate. For the heat equation, CN gave $O(k^2 + h^2)$ behaviour, so when k and h shrink together by the same factor the error drops by roughly 4. BD has only $O(k + h^2)$, so for the same shrink factor the temporal part of the error only halves.

Why does the Forward-Difference method become unstable when $\lambda > \frac{1}{2}$? The amplification factor for the n -th Fourier mode is $g_n = 1 - 4\lambda \sin^2(n\pi h/2)$. The most negative value occurs at the highest mode and equals $1 - 4\lambda$; stability requires $|g_n| \leq 1$, i.e. $\lambda \leq \frac{1}{2}$. Beyond this, the high mode is multiplied by a factor with $|g_n| > 1$ at every step, so any roundoff in that mode is amplified exponentially. In my run this produced a final $|w|_\infty$ of nearly 6×10^6 even though the exact peak at $t = 0.5$ is below 10^{-2} .

Why do implicit methods stay stable for larger time steps? For BD the amplification factor is $g_n = 1/(1 + 4\lambda \sin^2(n\pi h/2))$, which is in $(0, 1]$ for every $\lambda > 0$ and every n . For CN it is $(1 - 2\lambda \sin^2)/(1 + 2\lambda \sin^2)$, which lies in $(-1, 1]$ for every $\lambda > 0$. So both are unconditionally stable, and the time step is governed only by accuracy, not stability. That is what makes the implicit schemes the natural choice when you need to take large time steps or when the problem is stiff.

How does stability differ between elliptic and parabolic PDEs? The elliptic problem has no time stepping, so stability instead shows up as the question of whether the iterative solver converges and how fast. Gauss–Seidel converges for the five-point stencil because the matrix is strictly diagonally dominant (the row sum is 4 on the diagonal and at most 4 off the diagonal). For the parabolic problem, stability is about whether errors at one time step are amplified or damped by the marching operator, which is exactly the Fourier analysis above.

How do errors propagate over time? For Forward-Difference with $\lambda \leq \frac{1}{2}$ each mode is damped per step, so errors do not grow (and the right panel of Figure 2 shows the max error actually shrinks with time as the true solution itself decays). For BD and CN the amplification factor is also bounded by 1 in magnitude for every mode, so again errors do not grow. When the Forward stability constraint is violated the highest mode is amplified at every step, which is the standard divergence in the middle panel of Figure 2.

What tradeoffs exist? Explicit methods are cheap per step but require small k for stability, so they may end up doing more total work. Implicit methods cost a tridiagonal solve per step (linear in m via Crout, not $O(m^3)$), but they let you pick k for accuracy alone. CN buys an extra order of temporal accuracy over BD at the same per-step cost, so it is usually the right default for the heat equation unless something else (like non-smooth initial data) makes its slightly more oscillatory damping problematic.

8 Conclusions

The numerical experiments confirmed the theoretical pictures from class. The five-point stencil is $O(h^2)$ accurate on the Poisson check (Problem C2) and gave the exact answer on Problem A because that problem’s solution happens to be bilinear. The Forward-Difference scheme is conditionally stable exactly where $\lambda \leq \frac{1}{2}$, and crossing that threshold produces explosive growth in the highest representable mode. Crank–Nicolson is unconditionally stable and one order more accurate in time than Backward-Difference, which shows up as a roughly $10\times$ smaller error at the same h and k in Problem B. The multi-mode experiment in Problem C1 made the spectral structure of the heat equation directly visible: the $n = 3$ mode decayed nine times faster than the $n = 1$ mode, exactly as the analytic decay rates $e^{-n^2\pi^2t}$ predict.

9 Code

All solvers are in `pde_methods.py`, and `run_project.py` reproduces every figure and table in this report; `check_regression.py` runs sanity checks (Problem A reduces to $u = 100xy$, Poisson five-point stencil is $O(h^2)$, Crank–Nicolson reproduces the analytic decay, Forward-Difference blows up when $\lambda > \frac{1}{2}$). The code is available at:

<https://github.com/Hovatar/final-projects/tree/master/math104c-final-project-2>